ButterflyFX™ Dimensional Programming Engine

Governing Document

A Unified Specification for Manifold Computing

Author: Kenneth Bingham

Date: April 2026

Version: 1.0

Classification: Confidential

“Store Geometry, Not Bytes™”

ButterflyFX™ Governing Document v1.0 — Confidential — © 2026 Kenneth Bingham

Table of Contents

Chapter 1 — Preamble and Philosophy (Section 1.0)

1.1 Purpose

1.2 Foundational Philosophy

1.3 Core Axiom

Chapter 2 — The Algebra: Coordinate System and Manifold Arithmetic (Section 2.0)

2.1 The Five Primitives

2.2 Surface Equations

2.3 Completeness Principle

2.4 Arithmetic Operations

Figure 1: Manifold Topology

Chapter 3 — The Gyroid Substrate (Section 3.0)

3.1 What Is the Gyroid

3.2 Why the Gyroid

3.3 Substrate Mapping

3.4 Substrate Traversal

Figure 2: Gyroid Substrate — Dual Labyrinth Architecture

Chapter 4 — The Resonant Lifecycle (Section 4.0)

4.1 Overview

4.2 Lifecycle Phases

4.3 Resonance Frequency

4.4 Lifecycle Guarantees

Figure 3: The Resonant Lifecycle

Chapter 5 — Bloom and Collapse Mechanics (Section 5.0)

5.1 Bloom Defined

5.2 Bloom Rules

5.3 Collapse Defined

5.4 Collapse Rules

5.5 The Bloom/Collapse Balance

Figure 4: Bloom/Collapse Mechanics

Chapter 6 — Dimensional Flow (Section 6.0)

6.1 What Is Dimensional Flow

6.2 Flow Directions

6.3 Flow Constraints

6.4 Recursive Dimensional Nesting

Figure 5: Dimensional Flow

Chapter 7 — The Resolver Architecture (Section 7.0)

7.1 Purpose

7.2 Resolver Pipeline

7.3 Resolver Guarantees

7.4 Resolver Substrate

Figure 6: Resolver Architecture

Chapter 8 — State-Driven Runtime Model (Section 8.0)

8.1 Runtime Overview

8.2 State Machine

8.3 Transition Dispatch

8.4 Event Sourcing

8.5 No Imperative Control Flow

Chapter 9 — The Manifold Database (Section 9.0)

9.1 “Store Geometry, Not Bytes”

9.2 Schema as Surface

9.3 Query Model

9.4 Indexing via Gyroid

9.5 Persistence

Chapter 10 — Concurrency Model (Section 10.0)

10.1 Dual Labyrinth Isolation

10.2 Lock-Free Resonance

10.3 Bloom Concurrency

10.4 Collapse Ordering

10.5 Deadlock Freedom

Chapter 11 — Operational Directives (Section 11.0)

11.1 Directive 1 — Observation First

11.2 Directive 2 — Geometry Over Bytes

11.3 Directive 3 — Manifold Integrity

11.4 Directive 4 — Resonance Monitoring

11.5 Directive 5 — Collapse Safety

11.6 Directive 6 — Recursive Respect

11.7 Directive 7 — Concurrency Discipline

11.8 Directive 8 — Auditability

Appendix A — Glossary

Appendix B — Quick Reference: State Transition Matrix

Appendix C — Figure Index

Chapter 1 — Preamble and Philosophy

Section 1.0

1.1 — Purpose

This document is the single authoritative specification governing the ButterflyFX™ Dimensional Programming Engine. It defines the algebra, substrate, lifecycle, runtime model, database architecture, resolver mechanics, concurrency guarantees, and operational directives that together constitute the engine.

All implementations, extensions, integrations, and derivative works must conform to the principles, constraints, and guarantees set forth herein. Where ambiguity

exists, this document governs. Where silence exists, the foundational philosophy (Section 1.2) governs.

This specification is versioned. The current version is **1.0**, dated **April 2026**. All future amendments shall be issued as numbered revisions and appended to the revision history.

1.2 — Foundational Philosophy

FOUNDATIONAL AXIOM

"Everything is a point."

A line is points. A breadth is points. A depth is points. A volume is points. The paradigm is recursive — it starts again on a higher plane.

Any observed point has coordinates $m(x, y, z)$. Every x has its full complement of y, every y its full complement of z, every z and every m has its full complement of (x, y, z) — even if not manifest until observed.

PRINCIPLE: Potential Completeness with Actual Economy

"We do not build infinite grids; we only build what is needed — however, the infinite grid is there in spirit."

The manifold exists in totality but materializes only at observation. This is potential completeness with actual economy: every coordinate is addressable, but only observed coordinates consume resources.

This philosophy eliminates waste at the architectural level. Traditional computing systems allocate storage eagerly, consuming resources for data that may never be accessed. The ButterflyFX™ engine inverts this paradigm: the complete manifold is always available in potential, and materialization is strictly observation-driven.

1.3 — Core Axiom

CORE AXIOM: Shapes Are Data

Data need not be stored on a hard drive when they exist on the surface of geometry.

The foundational manifold surfaces $z = xy$ and $z = xy^2$ were discovered by Kenneth Bingham over a decade ago in Blender 3D and brought to full potential with the onset of AI. These two surfaces — the linear hyperbolic paraboloid and its quadratic counterpart — encode the complete state space of any computable system as geometry rather than bytes.

This is not metaphor. The engine literally stores data as coordinates on manifold surfaces. Retrieval is geometric projection. Indexing is geodesic proximity. Persistence is surface encoding. The traditional abstraction layers of file systems, serialization formats, and byte-addressable memory are replaced by a single, unified geometric substrate.

Chapter 2 — The Algebra

Coordinate System and Manifold Arithmetic — Section 2.0

2.1 — The Five Primitives

The ButterflyFX™ algebra is founded upon five primitives. These primitives are irreducible — all manifold operations, state representations, and computational expressions are composed from them.

Primitive	Name	Definition
x	Primary Axis	The first independent dimension. Represents domain, category, or identity. The x-axis is the foundational axis of classification — it answers the question *"what kind?"*
y	Secondary Axis	The second independent dimension. Represents attribute, property, or variation. The y-axis provides depth within a domain — it answers the question *"which aspect?"*
y^2	Quadratic Intensifier	y squared introduces curvature and nonlinear manifold behavior. It is the second foundational surface. Where y provides linear variation, y^2 provides exponential richness.
z	Dependent Axis	Computed as $z = xy$ (linear surface) or $z = xy^2$ (quadratic surface). z is always derived, never independent. It is the manifold's depth — the product of classification and attribute.
m	Manifold Point	$m(x, y, z)$ is the fully resolved coordinate. m encapsulates the complete state of any observed point on the manifold — it is the atomic unit of the engine.

2.2 — Surface Equations

The ButterflyFX™ engine operates on two foundational manifold surfaces:

LINEAR SURFACE

$$z = xy$$

A ruled hyperbolic paraboloid (saddle shape). Every point on this surface represents a data state whose depth is the product of its two independent coordinates. The linear surface is the engine's primary computational manifold — efficient, predictable, and algebraically transparent.

QUADRATIC SURFACE

$$z = xy^2$$

Introduces curvature; the depth grows quadratically with y, producing a richer, more expressive manifold suitable for complex state spaces. The quadratic surface is deployed when linear resolution is insufficient — when the problem domain demands nonlinear state differentiation.

2.3 — Completeness Principle

COMPLETENESS PRINCIPLE

Every x has its full complement of y. Every y has its full complement of z. Every m has its full complement of (x, y, z).

This is not built eagerly — it is latent. The manifold is complete in potential but economical in manifestation. Only observed points are materialized.

The completeness principle guarantees that no coordinate query will ever return "undefined" or "out of range." Every possible (x, y) pair has a corresponding z value, and every possible (x, y, z) triple constitutes a valid manifold point m. The

question is never *whether* a point exists, but whether it has been *observed* — that is, whether it is currently materialized or remains latent.

2.4 — Arithmetic Operations

Manifold arithmetic defines how points combine, project, and transform across the algebra:

Operation	Notation	Definition
Addition (Superposition)	$m_1 + m_2$	Superposition of two manifold points. The resulting point occupies the coordinate-wise sum: $m_3(x_1+x_2, y_1+y_2, z_1+z_2)$. Superposition expresses the co-existence of two states on the manifold.
Projection (Dimensional Collapse)	$\pi_d(m)$	Collapsing a dimension d from a manifold point. Projecting along y yields a point on the xz-plane; projecting along x yields a point on the yz-plane. Projection reduces dimensionality while preserving the remaining coordinates.
Transformation (Surface Mapping)	$T: S_1 \to S_2$	Mapping a point from one surface to another — e.g., from $z = xy$ to $z = xy^2$. The x and y coordinates are preserved; only z is recomputed under the target surface equation.

Figure 1: Manifold Topology — Linear ($z = xy$) and Quadratic ($z = xy^2$) Surfaces

Table A: Linear Surface — $z = xy$

$y_1 = 1$	z = 1	z = 2	z = 3	z = 4	z = 5

$y_2 = 2$	z = 2	z = 4	z = 6	z = 8	z = 10
$y_3 = 3$	z = 3	z = 6	z = 9	z = 12	z = 15
$y_4 = 4$	z = 4	z = 8	z = 12	z = 16	z = 20
$y_5 = 5$	z = 5	z = 10	z = 15	z = 20	z = 25

Table B: Quadratic Surface — $z = xy^2$

$y_1 = 1$	z = 1	z = 2	z = 3	z = 4	z = 5
$y_2 = 2$	z = 4	z = 8	z = 12	z = 16	z = 20
$y_3 = 3$	z = 9	z = 18	z = 27	z = 36	z = 45
$y_4 = 4$	z = 16	z = 32	z = 48	z = 64	z = 80
$y_5 = 5$	z = 25	z = 50	z = 75	z = 100	z = 125

Each cell represents a materializable manifold point $m(x, y, z)$. All cells shown are materialized for illustration; in practice, only observed cells are active. Unobserved cells remain latent — fully addressable but consuming no resources. Note the dramatic divergence between linear and quadratic surfaces at higher y values, demonstrating the expressive power of the quadratic intensifier.

ButterflyFX™ Governing Document v1.0 — Confidential — © 2026 Kenneth Bingham

---

Chapter 3 — The Gyroid Substrate

Section 3.0

3.1 — What Is the Gyroid

The Gyroid is a triply periodic minimal surface (TPMS) that serves as the computational substrate of the ButterflyFX™ engine. It possesses the following defining properties:

- **Zero mean curvature at every point** — the surface is a minimal surface, meaning it locally minimizes area. This property ensures that data traversal paths across the substrate are inherently optimized.
- **Infinite connectivity** — the surface is a single, connected sheet that extends infinitely in all three spatial dimensions without self-intersection.
- **Dual labyrinth structure** — the Gyroid divides three-dimensional space into two interpenetrating, non-intersecting labyrinths. Each labyrinth is itself infinitely connected.
- **Triple periodicity** — the surface repeats identically in all three coordinate directions, providing natural tiling and scalability.

THE GYROID EQUATION

$$\sin(x)\cos(y) + \sin(y)\cos(z) + \sin(z)\cos(x) = 0$$

3.2 — Why the Gyroid

The Gyroid was selected as the engine's substrate for four structural reasons:

ID	Property	Computational Benefit
(a)	**Infinite Connectivity**	Every region of the substrate is reachable from every other region without crossing a boundary. This guarantees that any manifold point can be resolved from any starting location.

ID	Property	Computational Benefit
(b)	**Minimal Surface Area**	Data traversal paths are inherently optimized. The surface minimizes the distance between any two connected points, reducing resolver latency.
(c)	**Self-Similarity**	The substrate is fractal-like, supporting recursive dimensional nesting. A region of the Gyroid at one scale mirrors the structure at any other scale, enabling the recursive paradigm described in Section 6.4.
(d)	**Natural Partitioning**	The two labyrinths provide inherent isolation for concurrent operations. Compute and storage can operate in separate labyrinths without synchronization overhead (Section 10.1).

3.3 — Substrate Mapping

Manifold points $m(x, y, z)$ are mapped onto the Gyroid surface through a bijective projection. Each point on the $z = xy$ or $z = xy^2$ surface is projected onto the nearest Gyroid surface point, establishing a one-to-one correspondence between algebraic coordinates and substrate locations.

The mapping preserves topological neighborhoods: points that are algebraically close (small differences in x, y, or z) map to Gyroid surface points that are geodesically close. This locality preservation is essential for efficient range queries and neighbor traversal in the manifold database (Chapter 9).

3.4 — Substrate Traversal

Navigation across the Gyroid follows geodesic paths — the shortest paths on the curved surface. The engine’s resolver (Chapter 7) uses these geodesics to locate

and connect manifold points. Because the Gyroid is a minimal surface, geodesic computation is efficient: the zero mean curvature ensures that geodesics do not oscillate or diverge unpredictably.

Figure 2: Gyroid Substrate — Dual Labyrinth Architecture

Active computation threads	$\sin(x)\cos(y) + \sin(y)\cos(z) + \sin(z)\cos(x) = 0$	Manifold point storage
Bloom operations	↔ Geodesic traversal paths ↔	Collapsed state archive
Resolver queries	↔ Bijective projection ↔	Materialized $m(x, y, z)$ points
Concurrent workers	↔ Isolation boundary ↔	Persistent geometry

The Gyroid surface serves as the boundary between two interpenetrating labyrinths. Labyrinth A hosts all active computation — bloom operations, resolver queries, and concurrent worker threads. Labyrinth B hosts all persistent state — materialized manifold points, collapsed state archives, and encoded geometry. The two labyrinths share the Gyroid surface but never cross it within a single operation, providing natural concurrency isolation (see Chapter 10).

ButterflyFX™ Governing Document v1.0 — Confidential — © 2026 Kenneth Bingham

---

Chapter 4 — The Resonant Lifecycle

Section 4.0

4.1 — Overview

The resonant lifecycle is the heartbeat of the ButterflyFX™ engine. Every manifold point undergoes a cyclical progression through eight defined states. The lifecycle is “resonant” because it is not linear — points oscillate between states, and the frequency of oscillation encodes computational intensity.

The lifecycle governs resource allocation, state persistence, and computational scheduling. It is the mechanism by which the engine achieves potential completeness with actual economy: points consume resources only while active, and return those resources upon collapse.

4.2 — Lifecycle Phases

#	Phase	Definition
1	**LATENT**	The point exists in potential but has not been observed. No resources are allocated. The coordinate is addressable but not materialized. This is the ground state of every manifold point.
2	**OBSERVED**	An external query or event targets this coordinate. The point begins the materialization process. Observation is the sole trigger for resource allocation — no point is ever materialized speculatively.
3	**MATERIALIZING**	Resources are allocated; the point is computed from the algebra ($z = xy$ or $z = xy^2$). The substrate mapping (Section 3.3) is executed, and the point is assigned a location on the Gyroid surface.

#	Phase	Definition
4	**ACTIVE**	The point is fully materialized on the Gyroid substrate, participating in computation. It holds allocated resources and occupies a defined substrate location. The point is available for read and write operations.
5	**RESONATING**	The point is in active use, oscillating between read and write operations. Resonance frequency is the rate of oscillation — a key metric for scheduling and resource management.
6	**DAMPENING**	Usage frequency decreases; the point begins preparing for collapse. Dampening is a gradual process — the point remains functional but signals to the runtime that its resources may soon be reclaimable.
7	**COLLAPSING**	Resources are being reclaimed; the point’s state is archived to geometry. The point’s current values are encoded onto the manifold surface as geometric coordinates. Active references are resolved or transferred.
8	**COLLAPSED**	The point returns to latent potential, its state encoded on the manifold surface. No active resources are consumed. The point can be re-observed and re-materialized at any time, restoring its full state

#	Phase	Definition
		from the geometric encoding.

4.3 — Resonance Frequency

Resonance frequency is the rate at which a point transitions between the ACTIVE and RESONATING states. It is measured in oscillations per unit time and serves as the engine's primary scheduling signal:

- **High-frequency resonance** indicates heavy computational demand. The runtime prioritizes high-frequency points for resource allocation and cache proximity on the Gyroid substrate.
- **Low-frequency resonance** indicates declining usage. The runtime initiates dampening procedures for low-frequency points, preparing them for eventual collapse.
- **Zero-frequency** indicates a point that has transitioned from ACTIVE to DAMPENING — it is no longer oscillating and is a candidate for collapse.

4.4 — Lifecycle Guarantees

GUARANTEE: No Point Is Ever Destroyed

The manifold is eternal. Collapse is archival, not deletion. Every collapsed point can be re-observed and re-materialized with its full state intact. The lifecycle is circular: COLLAPSED → LATENT. No information is lost at any phase transition.

Figure 3: The Resonant Lifecycle — Eight Phases of Manifold Point Evolution

① **LATENT**	→ Observation event triggers →
② **OBSERVED**	→ Resource allocation begins →
③ **MATERIALIZING**	→ $z = xy$ or $z = xy^2$ computed →
④ **ACTIVE**	→ Substrate placement complete →
⑤ **RESONATING**	↔ Read/Write oscillation ↔
⑥ **DAMPENING**	→ Frequency declining →
⑦ **COLLAPSING**	→ State archival to geometry →
⑧ **COLLAPSED**	→ Returns to LATENT (cycle repeats) →

NOTE

The lifecycle is circular: COLLAPSED → LATENT. No point is ever destroyed. The cycle may repeat indefinitely, with each iteration potentially producing different resonance characteristics based on current computational demand.

Chapter 5 — Bloom and Collapse Mechanics

Section 5.0

5.1 — Bloom Defined

Bloom is the controlled expansion of manifold state. When a region of the manifold is observed, its latent points bloom into active existence — transitioning from

LATENT through OBSERVED, MATERIALIZING, and into ACTIVE. Bloom is the engine's mechanism for demand-driven resource allocation.

Bloom is recursive: observing a point at one dimensional level may trigger bloom at higher or lower levels. A single observation event can cascade through the manifold, materializing an entire neighborhood of related points. This cascade is bounded by configurable parameters to prevent unbounded expansion.

5.2 — Bloom Rules

Rule	Name	Specification
B1	Observation-Only Trigger	Bloom is triggered only by observation — never speculatively. No point may bloom without an explicit observation event targeting its coordinate or a geodesic neighbor's coordinate.
B2	Geodesic Propagation	Bloom propagates along the Gyroid geodesic to adjacent latent points within a defined radius. The propagation radius is configurable per manifold region.
B3	Bounded Depth	Bloom depth is bounded by a configurable parameter (**max_bloom_depth**) to prevent unbounded expansion. Default: 3 levels.
B4	Dimensional Isolation	Bloom respects dimensional boundaries — blooming in x does not automatically bloom in y or z. Cross-dimensional bloom requires explicit observation on each axis.
B5	Nested Bloom	Nested bloom (bloom triggering bloom) is permitted up to

Rule	Name	Specification
		max_bloom_depth levels. Each nested level operates as an independent bloom event with its own propagation radius.

5.3 — Collapse Defined

Collapse is the inverse of bloom — the controlled contraction of active state back into latent geometry. Collapse encodes the point's current state onto the manifold surface, preserving it without consuming active resources. Collapse is the engine's mechanism for resource reclamation.

Collapse is not deletion. The collapsed point's state is fully preserved as a geometric encoding on the manifold surface. Re-observation of the coordinate will trigger re-materialization with the complete archived state restored.

5.4 — Collapse Rules

Rule	Name	Specification
C1	Dampening Trigger	Collapse is triggered by dampening below a threshold resonance frequency. The threshold is configurable per manifold region and adapts to system resource pressure.
C2	Leaf-to-Root Ordering	Collapse proceeds leaf-to-root — innermost nested points collapse first. This ensures that parent points are never orphaned by premature child collapse.
C3	Reference Integrity	A point cannot collapse while any dependent computation references

Rule	Name	Specification
		it. Reference counting ensures that active dependencies prevent premature collapse. **This rule is never violated, even under emergency conditions.**
C4	Geometric Encoding	Collapsed state is encoded geometrically on the manifold surface — no byte-level storage. The encoding preserves full fidelity of the point's state at the moment of collapse.
C5	Emergency Collapse	Emergency collapse (triggered by system resource pressure) may override normal dampening timelines, but Rule C3 is never violated. Emergency collapse accelerates timing but does not skip reference integrity checks.

5.5 — The Bloom/Collapse Balance

The engine maintains a dynamic equilibrium between bloom and collapse. The ratio of active-to-latent points is a key health metric, monitored continuously by the runtime. An imbalance in either direction signals a problem:

- **Excessive bloom** (too many active points) — indicates resource overconsumption. The runtime increases dampening pressure to accelerate collapse of low-frequency points.
- **Excessive collapse** (too few active points) — indicates underutilization. The runtime relaxes dampening thresholds to allow longer resonance periods.

Figure 4: Bloom/Collapse Mechanics — Symmetric State Transitions

Encode to geometry	**COLLAPSED ◄—— ——► LATENT**	Awaiting observation
Reference count check	**COLLAPSING ◄—— —— ► OBSERVED**	Observation received
Frequency declining	**DAMPENING ◄—— ——► MATERIALIZING**	Resources allocating
Oscillation slowing	**RESONATING ◄═══► ACTIVE**	Fully materialized
◄— EQUILIBRIUM ZONE —►		

The engine maintains dynamic equilibrium. Bloom expands rightward — from LATENT through OBSERVED, MATERIALIZING, and into ACTIVE/RESONATING. Collapse contracts leftward — from DAMPENING through COLLAPSING and into COLLAPSED (which returns to LATENT). The ACTIVE/RESONATING zone is the equilibrium center where computation occurs.

ButterflyFX™ Governing Document v1.0 — Confidential — © 2026 Kenneth Bingham

---

Chapter 6 — Dimensional Flow

Section 6.0

6.1 — What Is Dimensional Flow

Dimensional flow describes how data and computation move across the manifold’s dimensional axes. Flow is not arbitrary — it follows the manifold surface, constrained by the algebra. All movement of data within the engine is expressed as flow along one or more axes, governed by the surface equations and substrate topology.

6.2 — Flow Directions

Flow Type	Axis	Description
X-Flow	Primary (x)	Movement along the primary axis. Traverses categories, domains, or identities. X-Flow moves across the breadth of the manifold — it changes *what kind* of data is being accessed while holding attribute and depth constant.
Y-Flow	Secondary (y)	Movement along the secondary axis. Traverses attributes, properties, or variations. Y-Flow moves within a domain — it changes *which aspect* is being accessed while holding identity and (potentially) depth constant.
Z-Flow	Dependent (z)	Movement along the dependent axis. Always derived — Z-Flow is a consequence of X-Flow and Y-Flow combined. As x and y change, z changes according to the surface equation. Z-Flow cannot be initiated independently.
M-Flow	Cross-Manifold	Cross-manifold movement. A point on one manifold surface ($z = xy$) may project onto another ($z = xy^2$). M-Flow is a transformation (Section 2.4) that preserves x and y while recomputing z under a different surface equation.

6.3 — Flow Constraints

All flow is subject to the following constraints:

1. **Surface Adherence:** Flow must respect the manifold surface — no point can exist off-surface. All paths traced by flow must lie entirely on $z = xy$ or $z = xy^2$.
2. **Curvature-Dependent Velocity:** Flow velocity is proportional to the surface curvature at the current point. Flatter regions permit faster flow; regions of higher curvature (particularly on the quadratic surface) impose slower flow. This is a natural throttling mechanism.
3. **Geodesic Routing:** Flow follows geodesic paths on the Gyroid substrate. The shortest path between two points on the substrate determines the flow route.

6.4 — Recursive Dimensional Nesting

RECURSIVE PARADIGM

The paradigm is recursive — at any dimensional level, the entire structure can restart on a higher plane.

A manifold point $m(x, y, z)$ can itself serve as the x-coordinate of a higher-dimensional manifold, producing $m'(m, y', z')$ — a point whose primary axis is itself a complete manifold.

Recursive nesting enables fractal-depth computation. A first-level manifold encodes primary data. A second-level manifold, whose x-axis consists of first-level manifold points, encodes meta-data. A third-level manifold encodes meta-meta-data. This nesting is unbounded in principle but bounded in practice by the **max_bloom_depth** parameter.

Figure 5: Dimensional Flow — Axes, Traversal, and Rules

x (Primary)	→ Horizontal	Domains, Categories, Identities	Independent — free traversal
y (Secondary)	↑ Vertical	Attributes, Properties, Variations	Independent — free traversal
y^2 (Quadratic)	↑↑ Curved Vertical	Nonlinear state intensification	Curvature-dependent velocity
z (Dependent)	↗ Derived Diagonal	Computed depth: $z = xy$ or $z = xy^2$	Always derived — never independent
m (Manifold Point)	● Point	Complete state: $m(x, y, z)$	Observation-gated materialization
m' (Recursive)	⟳ Higher Plane	m as x-coordinate of next level	Recursive nesting — fractal depth

Dimensional flow is constrained by the algebra and the substrate. The x and y axes permit free traversal; the z axis is always derived. The recursive level (m') enables fractal nesting, where an entire manifold serves as a single coordinate on a higher-dimensional manifold.

ButterflyFX™ Governing Document v1.0 — Confidential — © 2026 Kenneth Bingham

---

Chapter 7 — The Resolver Architecture

Section 7.0

7.1 — Purpose

The Resolver is the engine component responsible for locating, materializing, and connecting manifold points. Given a coordinate query $m(x, y, z)$, the Resolver finds the corresponding point on the Gyroid substrate or triggers its bloom. The

Resolver is the interface between the algebraic abstraction (coordinates) and the physical substrate (Gyroid surface points).

Every data access, every computation initiation, and every state query in the ButterflyFX™ engine passes through the Resolver. It is the single point of contact between the logical manifold and the physical substrate.

7.2 — Resolver Pipeline

The Resolver operates as a six-stage pipeline. Each stage is deterministic, and the pipeline as a whole is idempotent and non-blocking.

Stage	Name	Operation
1	PARSE	Decompose the incoming query into its constituent (x, y, z) components. Validate that the coordinates are well-formed and within the manifold's addressable range.
2	COMPUTE	Calculate z from the algebra. If z is not provided in the query, compute it as $z = xy$ (linear) or $z = xy^2$ (quadratic) based on the target surface. If z is provided, validate it against the algebra.
3	PROJECT	Map the algebraic coordinate (x, y, z) onto the Gyroid substrate. Compute the nearest Gyroid surface point via bijective projection (Section 3.3). Output: substrate coordinates.
4	LOCATE	Search the Gyroid substrate for an existing materialized point at the projected coordinates. Output: HIT (point found and active) or MISS

Stage	Name	Operation
		(point is latent).
5	BLOOM-OR-RETURN	If LOCATE returned HIT, return the existing materialized point. If LOCATE returned MISS, trigger bloom at the target coordinate and await materialization. Output: the fully materialized $m(x, y, z)$.
6	BIND	Attach the resolved point to the requesting computation. Increment the point's reference count to prevent premature collapse (Rule C3). Output: a bound reference.

7.3 — Resolver Guarantees

RESOLVER GUARANTEES

- **Idempotent:** Resolving the same coordinate always returns the same point (or triggers its re-materialization identically). Multiple resolutions of the same coordinate produce identical results.
- **Deterministic:** The algebraic computation is pure — same inputs always produce the same z. There is no randomness, no side-channel input, and no non-determinism in the Resolver pipeline.
- **Non-blocking:** The Resolver pipeline is asynchronous. Multiple resolutions proceed concurrently without mutual exclusion. No resolution operation blocks any other resolution operation.

7.4 — Resolver Substrate

The Resolver itself operates on the Gyroid substrate, using the dual labyrinth architecture for its own internal operations. Labyrinth A hosts query processing — the PARSE, COMPUTE, PROJECT, and LOCATE stages. Labyrinth B hosts result caching — previously resolved coordinates are stored as Gyroid surface encodings for rapid retrieval on subsequent queries.

Figure 6: Resolver Architecture — Six-Stage Pipeline

Stage 1: PARSE	Decompose query into (x, y, z) components	Raw coordinates
Stage 2: COMPUTE	Apply algebra: $z = xy$ or $z = xy^2$	Resolved z value
Stage 3: PROJECT	Map onto Gyroid substrate surface	Substrate coordinates
Stage 4: LOCATE	Search for existing materialized point	HIT (point) or MISS (latent)
Stage 5: BLOOM-OR-RETURN	If HIT → return point; If MISS → trigger bloom	Materialized $m(x, y, z)$
Stage 6: BIND	Attach resolved point to requesting computation	Bound reference

Pipeline Guarantees: Idempotent | Deterministic | Non-blocking | Asynchronous

Chapter 8 — State-Driven Runtime Model

Section 8.0

8.1 — Runtime Overview

The ButterflyFX™ engine's runtime is entirely state-driven. There is no imperative control flow — no sequential instruction execution, no program counter, no call stack in the traditional sense. Computation advances by state transitions on manifold points. The runtime observes the manifold, detects state changes, and dispatches handlers.

The runtime is, in essence, a global observer of the manifold. It watches for phase transitions in the resonant lifecycle and responds to each transition by executing the corresponding handler. The manifold itself is both the program and the data.

8.2 — State Machine

Every manifold point carries a state drawn from the eight lifecycle phases defined in Section 4.2 (LATENT through COLLAPSED). The runtime maintains a global state machine that encompasses all active points on the manifold. The state machine's transition function is defined by the lifecycle rules and the bloom/collapse mechanics:

Current State	Valid Transitions	Transition Condition
LATENT	→ OBSERVED	External observation event received
OBSERVED	→ MATERIALIZING	Resources available; allocation initiated
MATERIALIZING	→ ACTIVE	Algebra computed; substrate mapping complete

Current State	Valid Transitions	Transition Condition
ACTIVE	→ RESONATING	First read/write operation performed
RESONATING	→ DAMPENING	Resonance frequency drops below threshold
DAMPENING	→ COLLAPSING	Dampening period elapsed; collapse initiated
DAMPENING	→ RESONATING	New read/write restores frequency (re-excitation)
COLLAPSING	→ COLLAPSED	All references released; state archived to geometry
COLLAPSED	→ LATENT	Resources fully reclaimed; returns to ground state

8.3 — Transition Dispatch

When a point transitions state, the runtime dispatches the appropriate handler from the following handler registry:

- **Observation Handler:** Invoked on LATENT → OBSERVED. Validates the observation event, allocates a materialization slot, and enqueues the point for computation.
- **Materialization Handler:** Invoked on OBSERVED → MATERIALIZING. Executes the algebraic computation, performs substrate mapping, and transitions the point to ACTIVE upon completion.
- **Resonance Handler:** Invoked on ACTIVE → RESONATING (and on each subsequent oscillation). Monitors resonance frequency and reports metrics to the scheduling system.
- **Dampening Handler:** Invoked on RESONATING → DAMPENING. Marks the point as a collapse candidate and begins the dampening countdown.

- **Collapse Handler:** Invoked on DAMPENING → COLLAPSING. Checks reference counts (Rule C3), encodes state to geometry (Rule C4), and releases resources upon completion.

8.4 — Event Sourcing

All state transitions are recorded as events on the manifold surface itself. The geometry encodes its own history — every transition a point has undergone is preserved as part of the manifold's geometric encoding. This is event sourcing without a traditional event store: the manifold IS the event log.

To replay the history of any manifold point, one need only read the geometric encoding at its coordinate. The sequence of transitions, timestamps, and associated data are all encoded in the surface geometry.

8.5 — No Imperative Control Flow

> **PARADIGM: REACTIVE MANIFOLD COMPUTING**
>
> There are no loops, branches, or function calls in the traditional sense. All computation is expressed as manifold observations and state transitions. The runtime is reactive — it responds to state changes on the manifold rather than executing a predetermined sequence of instructions.

Traditional programs say "do this, then do that." The ButterflyFX™ engine says "when this state is observed, respond with that transition." The manifold drives computation; the runtime merely observes and dispatches.

Chapter 9 — The Manifold Database

Section 9.0

9.1 — “Store Geometry, Not Bytes”

DATABASE AXIOM

“Store Geometry, Not Bytes.™”

The manifold database stores all data as geometry on manifold surfaces. There are no tables, rows, or columns in the traditional sense. Data lives on the surface of $z = xy$ and $z = xy^2$. Queries are coordinate lookups. Joins are geodesic traversals.

The manifold database eliminates the impedance mismatch between application logic and storage. In traditional systems, data is serialized into bytes, stored on disk, deserialized into objects, manipulated, re-serialized, and re-stored. Each serialization/deserialization cycle introduces overhead, potential data loss, and conceptual friction. The manifold database operates directly on geometric coordinates — the data as stored IS the data as used.

9.2 — Schema as Surface

A traditional database schema maps directly to a manifold surface configuration:

Traditional Concept	Manifold Equivalent	Explanation
Table	x-axis (domain)	Each table becomes a position along the primary axis. Tables are domains — categories of

Traditional Concept	Manifold Equivalent	Explanation
		data distinguished by their x-coordinate.
Column	y-axis (attribute)	Each column becomes a position along the secondary axis. Columns are attributes — properties within a domain distinguished by their y-coordinate.
Cell Value	z-coordinate (depth)	Each cell value is a z-coordinate, computed from the algebra. The value IS the position on the manifold surface.
Row	Y-slice at fixed x	A row is a traversal along the y-axis at a fixed x-coordinate — all attributes of a single domain entity.
Index	Gyroid proximity	Indexing is geodesic proximity on the Gyroid substrate. Close coordinates are close on the surface.
Join	Geodesic traversal	Joins are geodesic paths between points on the substrate. Related data is connected by surface paths.

The schema IS the geometry. There is no separate schema definition language — the manifold surface equations and coordinate assignments constitute the schema.

9.3 — Query Model

Queries are coordinate resolutions. A query specifies an (x, y) pair, and the Resolver (Chapter 7) computes z and returns the manifold point:

Query Type	Example	Resolution
Point Lookup	$m(x_3, y_7, z)$	Linear: $z = 3 \times 7 = 21$.

Query Type	Example	Resolution
		Quadratic: $z = 3 \times 49 = 147$. Returns the manifold point at the computed depth.
Range Query	$m(x_1..x_5, y_3, z)$	Resolves all points along the x-axis at y_3. Geodesic neighborhood search on the Gyroid substrate.
Cross-Surface	T: $m_{lin} \rightarrow m_{quad}$	Projects a point from the linear surface to the quadratic surface. x and y preserved; z recomputed.

9.4 — Indexing via Gyroid

The Gyroid substrate serves as a natural spatial index. Because the bijective projection (Section 3.3) preserves topological neighborhoods, points that are algebraically close (similar x, y, z values) are also close on the Gyroid surface. This enables efficient range queries via geodesic neighborhood search — the substrate IS the index.

9.5 — Persistence

Collapsed points persist as geometric encodings on the manifold surface. No serialization to bytes occurs. Persistence IS the manifold. When a point collapses (Section 5.3), its complete state is encoded as geometric coordinates on the surface. When re-observed, the state is restored from the encoding with full fidelity.

There is no write-ahead log, no transaction journal, and no checkpoint mechanism in the traditional sense. The manifold's geometric encoding is inherently persistent — the surface exists whether or not the runtime is active.

Chapter 10 — Concurrency Model

Section 10.0

10.1 — Dual Labyrinth Isolation

The Gyroid’s two interpenetrating labyrinths provide natural isolation for concurrent operations. The assignment is fixed and invariant:

Labyrinth	Role	Operations
Labyrinth A	Compute	All active computation threads, bloom operations, resolver queries, and concurrent workers execute in Labyrinth A.
Labyrinth B	Storage	All manifold point storage, collapsed state archives, materialized point persistence, and geometric encodings reside in Labyrinth B.
Gyroid Surface	Boundary	The Gyroid surface itself serves as the communication boundary. Data passes between labyrinths only through surface projections. No operation crosses the boundary within a single execution context.

This dual labyrinth architecture eliminates the need for traditional locking mechanisms. Compute operations never directly access storage; they project onto the Gyroid surface and read from the surface projection. Storage operations never directly interact with computation; they receive surface projections and encode them.

10.2 — Lock-Free Resonance

Points in the RESONATING state oscillate between read and write operations without locks. The manifold algebra guarantees that no two independent points share the same (x, y, z) coordinate — every point occupies a unique position on the manifold surface. This natural uniqueness eliminates write conflicts: two concurrent operations targeting different coordinates can never interfere with each other.

For operations targeting the same coordinate, the Resolver's idempotency guarantee (Section 7.3) ensures consistent results regardless of ordering.

10.3 — Bloom Concurrency

Multiple bloom operations may proceed concurrently in different regions of the manifold. The Gyroid's infinite connectivity ensures that each bloom region has sufficient substrate space. Bloom operations in overlapping regions — where the geodesic propagation radius (Rule B2) of two concurrent blooms intersects — are serialized at the geodesic boundary to prevent duplicate materialization.

10.4 — Collapse Ordering

Collapse follows a strict leaf-to-root ordering, ensuring that no parent point collapses while children are still active. This ordering is enforced by reference counting (Rule C3): a parent point's reference count includes references from all child points. The parent cannot reach zero references (and thus cannot collapse) until all children have released their references by completing their own collapse.

10.5 — Deadlock Freedom

THEOREM: Deadlock Freedom

The combination of the following three properties guarantees freedom from deadlock in the ButterflyFX™ engine:

1. **Natural coordinate uniqueness** — no two operations contend for the same resource unless they target the same manifold point, in which case the Resolver's idempotency resolves the contention without blocking.
2. **Dual labyrinth isolation** — compute and storage never cross the Gyroid boundary within a single operation, eliminating cross-domain circular dependencies.
3. **Leaf-to-root collapse ordering** — collapse operations form a strict partial order (from leaves to root), preventing circular wait conditions.

Chapter 11 — Operational Directives

Section 11.0

The following eight directives are binding on all implementations, extensions, and operational deployments of the ButterflyFX™ Dimensional Programming Engine. Violation of any directive constitutes a non-conforming implementation.

11.1 — Directive 1: Observation First

DIRECTIVE D1 — OBSERVATION FIRST

Never materialize speculatively. All materialization is observation-driven. No manifold point may transition from LATENT to OBSERVED without an explicit,

traceable observation event. Speculative materialization — allocating resources for points that have not been observed — is prohibited.

11.2 — Directive 2: Geometry Over Bytes

DIRECTIVE D2 — GEOMETRY OVER BYTES

All persistence, indexing, and traversal operations use geometric representations. Byte-level serialization is prohibited except at system boundaries (I/O interfaces with external systems). Within the engine, all data exists as coordinates on manifold surfaces — never as serialized byte sequences.

11.3 — Directive 3: Manifold Integrity

DIRECTIVE D3 — MANIFOLD INTEGRITY

The manifold must remain topologically consistent at all times. No operation may: (a) create a point off-surface; (b) violate the algebra (z must equal xy or xy^2); or (c) break Gyroid connectivity. Any operation that would compromise manifold integrity must be rejected before execution.

11.4 — Directive 4: Resonance Monitoring

DIRECTIVE D4 — RESONANCE MONITORING

The runtime must continuously monitor resonance frequencies across all active points. When system resources are constrained, the runtime must

initiate dampening on the lowest-frequency points to free resources for higher-priority computation. Monitoring must be continuous — periodic polling is insufficient.

11.5 — Directive 5: Collapse Safety

DIRECTIVE D5 — COLLAPSE SAFETY

Collapse must never violate reference integrity (Rule C3). A point with active references must not collapse under any circumstance — including emergency resource pressure. Emergency collapse may accelerate the dampening timeline but must never skip reference counting or integrity checks.

11.6 — Directive 6: Recursive Respect

DIRECTIVE D6 — RECURSIVE RESPECT

When computation nests recursively — with a manifold point m serving as the x'-coordinate on a higher plane — each recursive level operates as an independent manifold with its own lifecycle, resolver, and resource allocation. Parent manifolds must not interfere with child manifold operations, and child manifolds must not access parent manifold state except through proper M-Flow channels.

11.7 — Directive 7: Concurrency Discipline

DIRECTIVE D7 — CONCURRENCY DISCIPLINE

Compute in Labyrinth A. Store in Labyrinth B. Never cross the Gyroid boundary within a single operation. All data exchange between labyrinths occurs through surface projection onto the Gyroid boundary. Any operation that directly accesses the opposing labyrinth is a conformance violation.

11.8 — Directive 8: Auditability

DIRECTIVE D8 — AUDITABILITY

All state transitions are events on the manifold. The geometry IS the audit log. Every phase transition, every bloom event, every collapse event, and every resolver query must be recorded as a geometric encoding on the manifold surface. No external logging system is required or permitted as the primary audit mechanism — the manifold is self-documenting.

Appendix A — Glossary

Term	Definition
Manifold	A mathematical surface defined by an equation (e.g., $z = xy$) that serves as the computational and data-storage medium of the engine. The manifold is the foundational abstraction — all data, computation, and state exist as points

Term	Definition
	on its surface.
Gyroid	A triply periodic minimal surface (TPMS) defined by $sin(x)cos(y) + sin(y)cos(z) + sin(z)cos(x) = 0$. The Gyroid serves as the engine's computational substrate, providing infinite connectivity, minimal surface area, self-similarity, and dual labyrinth isolation.
Substrate	The physical (or logical) surface on which manifold points are placed and traversed. In ButterflyFX™, the substrate is the Gyroid.
Bloom	The controlled expansion of manifold state — the process by which latent points are materialized into active existence in response to observation. Bloom is the engine's resource-allocation mechanism.
Collapse	The controlled contraction of active state back into latent geometry — the inverse of bloom. Collapse encodes point state onto the manifold surface and reclaims active resources.
Resonance	The oscillation of a manifold point between read and write operations while in the ACTIVE/RESONATING phase. Resonance frequency encodes computational intensity.
Dampening	The gradual decline in resonance frequency that precedes collapse. A dampening point is preparing to archive its state and release resources.
Latent	The ground state of a manifold point — existing in potential but not materialized. No resources are allocated. The point is addressable but not active.
Observation	An external event (query, access, reference) that targets a manifold coordinate. Observation is the sole trigger for materialization — the engine's equivalent of "first touch."
Materialization	The process of computing a manifold

Term	Definition
	point from the algebra and placing it on the Gyroid substrate. Materialization allocates resources and transitions the point from OBSERVED to ACTIVE.
Resolver	The engine component responsible for locating, materializing, and connecting manifold points. The Resolver is a six-stage pipeline: PARSE, COMPUTE, PROJECT, LOCATE, BLOOM-OR-RETURN, BIND.
Geodesic	The shortest path between two points on a curved surface. On the Gyroid substrate, geodesics are the natural traversal paths for resolver queries and dimensional flow.
Labyrinth	One of the two interpenetrating, non-intersecting volumes created by the Gyroid surface. Labyrinth A hosts computation; Labyrinth B hosts storage.
Surface	The two-dimensional boundary defined by a manifold equation. Data exists on the surface — neither above nor below it. The surfaces $z = xy$ and $z = xy^2$ are the engine's foundational surfaces.
$m(x, y, z)$	A fully resolved manifold point. The atomic unit of the ButterflyFX™ engine, encapsulating identity (x), attribute (y), and computed depth (z).
$z = xy$	The linear surface equation. Produces a ruled hyperbolic paraboloid (saddle shape). The engine's primary computational surface.
$z = xy^2$	The quadratic surface equation. Introduces curvature and nonlinear depth growth. Used for complex state spaces requiring greater expressive resolution.
TPMS	Triply Periodic Minimal Surface. A surface that is periodic in three independent directions and has zero mean curvature at every point. The Gyroid is an example of a TPMS.

Term	Definition
Coordinate	An (x, y, z) triple that identifies a unique point on the manifold surface. Coordinates are the address space of the engine.
Dimensional Flow	The movement of data and computation across the manifold's dimensional axes. Flow follows the manifold surface and is constrained by the algebra.
Recursive Nesting	The use of a manifold point m as the x-coordinate of a higher-dimensional manifold, enabling fractal-depth computation. Each recursive level is an independent manifold.

ButterflyFX™ Governing Document v1.0 — Confidential — © 2026 Kenneth Bingham

---

Appendix B — Quick Reference: State Transition Matrix

The following matrix defines all valid (✓) and invalid (✗) state transitions for manifold points. Valid transitions include the triggering condition.

LATENT	✗	✓ Observation event	✗	✗	✗	✗	✗	✗
OBSERVED	✗	✗	✓ Resources	✗	✗	✗	✗	✗

			allocated					
MATERIALIZING	✗	✗	✗	✓ Algebra computed	✗	✗	✗	✗
ACTIVE	✗	✗	✗	✗	✓ First R/W operation	✗	✗	✗
RESONATING	✗	✗	✗	✗	✗	✓ Freq. below threshold	✗	✗
DAMPENING	✗	✗	✗	✗	✓ Re-excitation (new R/W)	✗	✓ Dampening elapsed	✗
COLLAPSING	✗	✗	✗	✗	✗	✗	✗	✓ Refs released; archived
COLLAPSED	✓ Resources reclaimed	✗	✗	✗	✗	✗	✗	✗

NOTE

The only non-sequential transition is DAMPENING → RESONATING (re-excitation), which occurs when a dampening point receives new read/write activity, restoring its resonance frequency above the collapse threshold. All other transitions follow the strict lifecycle sequence. The lifecycle is circular:

COLLAPSED → LATENT completes the cycle.

Appendix C — Figure Index

Figure	Title	Section
Figure 1	Manifold Topology — Linear ($z = xy$) and Quadratic ($z = xy^2$) Surfaces	Section 2.4
Figure 2	Gyroid Substrate — Dual Labyrinth Architecture	Section 3.4
Figure 3	The Resonant Lifecycle — Eight Phases of Manifold Point Evolution	Section 4.4
Figure 4	Bloom/Collapse Mechanics — Symmetric State Transitions	Section 5.5
Figure 5	Dimensional Flow — Axes, Traversal, and Rules	Section 6.4
Figure 6	Resolver Architecture — Six-Stage Pipeline	Section 7.4

— END OF DOCUMENT —

"Store Geometry, Not Bytes™"

ButterflyFX™ Dimensional Programming Engine

Governing Document v1.0

April 2026